

Emergency Dispatch Priority and Coordination System

Project Report

ENSE707 Software Quality Assurance

Leo Cao (23211788), Fateh Bhular (23217987), Shawn Lee (23204035)

Auckland University of Technology

Table of Content

Task 1. Problem Definition and Approval.....	3
1.1 Background, Context and Stakeholders.....	3
1.2 Problem Statement and Software Quality Issues.....	4
1.3 Why the problem is suitable for a Software Quality Assurance Project.....	5
Task 2: Requirements and Quality Analysis.....	6
2.1 Functional Requirements.....	6
2.2 Non-Functional Requirements.....	7
2.3 Analysing Requirements Verification.....	10
Task 3: Proposed Solution and Initial Prototype.....	14
Task 4: AI-Assisted Development Using GitHub Copilot.....	15
Task 5: Initial Test Strategy and Test Planning.....	16
Task 6: Initial Test Cases and Requirements Traceability.....	17
Task 7: Project Progress, Risks, and Next Steps.....	18

Task 1. Problem Definition and Approval

1.1 Background, Context and Stakeholders

The emergency dispatch systems answer emergency calls, record details about the emergencies, and dispatch the right units accordingly. This project is inspired by real-world emergency dispatch; however, it is a simplified hypothetical model that does not represent an actual system. This will be used throughout this project.

The main stakeholders in the modelled emergency dispatch system are the **Caller**, **Dispatcher**, **Field Responders**, and **Software Developers**. The dispatch process begins with the Caller, who contacts the emergency service and provides information about the situation. The Dispatcher receives this information, records the emergency as a case, manually assigns its severity or priority, and determines which response units are required. The dispatcher then directly assigns available Field Responders, including ambulance, police, and fire units, to the case. Software Developers are also stakeholders as they are responsible for maintaining, testing, and improving the dispatch system.

The emergency dispatch software has significant practical relevance. Most importantly, a dispatch system's quality can have an impact on the effectiveness of an emergency. Failing to respond to a case punctually, assigning the wrong priority/severity, or poor coordination of response units could lead to slower emergency responses. Because emergency dispatching is a life-critical process, the software that supports it must operate accurately and consistently under pressure.

1.2 Problem Statement and Software Quality Issues

Managing emergency dispatching requires all aspects of the system to be of high quality because, unlike other systems, the emergency dispatch system is life-critical and requires immediate action. Any compromises in quality, such as issues in **reliability, usability, or maintainability**, could potentially come with the loss of lives.

The modelled emergency dispatch system relies on manual priority allocation by the dispatcher for incoming cases. This manual assignment could cause inconsistencies in priorities, resulting in low-priority cases being handled first and potential delays, **compromising the reliability** of the system.

For systems at the scale of emergency dispatching, it becomes difficult to control, especially since there are thousands of calls daily. Within the modelled system, dispatchers do not have an efficient way to record and manage call logs. The dispatcher also has to directly communicate and manually dispatch units to an ongoing case. At the scale of an emergency dispatching system, the number of requests for units becomes overwhelming and hard to manage, **compromising usability**.

The modelled system does not automatically log calls, meaning dispatchers need to do it manually, which takes time during the call. The current system for callers to dispatchers sometimes goes down for a few seconds at a time during the day, which **undermines reliability** since some calls may not come through.

The current system consists of multiple dispatchers all working towards assigning cases to dispatched units. However, this is a **reliability issue** as it opens up potential race condition vulnerabilities where a dispatched unit may be assigned to multiple cases at once.

Every emergency dispatch system needs to be **robust and maintainable**, due to the constant need for system updates. The proposed project's current design has three dispatchable units, these being ambulance, police, and fire units. If we were to add a new unit to our system, such as search and rescue, the current system is not modular and would not be able to accommodate it, requiring large refactoring. Maintainability is an important software quality issue to consider in the design phase since neglecting it increases the complexity of adding new unit types into the system. This, in turn, increases the likelihood of defects being introduced when the system is modified.

1.3 Why the problem is suitable for a Software Quality Assurance Project

Our selected problem is suitable for a Software Quality Assurance project because it contains clear software quality issues that can be analysed and tested. The main quality issues we mentioned in Section 1.2 are reliability, usability, and maintainability, which are also recognised quality characteristics within **ISO/IEC 25010**.

The **reliability issues** include possible race conditions during unit assignment, inconsistent priority allocation, and downtime within the modelled system. The **usability issues** come from the amount of manual work required from dispatchers, including recording calls and coordinating units under a high workload. The **maintainability issue** comes from the structure of the system being non-modular, where adding new units would require large changes to the existing code.

These issues are suitable for quality assurance because they can be easily used to form clear requirements, acceptance criteria, and test cases. This makes it possible to test the system to see if the quality issues have been improved or removed.

Quality assurance is incredibly important for an emergency dispatch system as defects can result in significant consequences such as delayed responses, misprioritisation of cases, or dispatching of units to the wrong case. Thus, before we can use a system, it must be reliable, usable, and maintainable.

- ☒ Identify initial functional requirements for the proposed solution.
- ☒ Identify initial non functional requirements for the proposed solution.
- ☒ Analyse requirements for clarity, completeness, consistency, correctness, feasibility, and testability.
- ☐ Rewrite or improve selected requirements to make them more testable.
- ☐ Define acceptance criteria for the key features.
- ☐ Identify relevant software quality attributes, such as usability, reliability, maintainability, security, performance, accessibility, compatibility, or portability.

Task 2: Requirements and Quality Analysis

2.1 Functional Requirements

- **FR-01** The system shall provide an interface for the dispatcher to record call details, including caller information, incident notice, description of incident, and timestamp.
- **FR-02** The system shall allow the dispatcher to assign a case from a call.
- **FR-03** The system shall manage all dispatched units with each unit's current availability status (not available/available), unit type, location, identifier, and unit personnel count.
- **FR-04** The system shall automatically assign a priority level to each incoming emergency case based on its severity.
- **FR-05** The system shall automatically route a case to the appropriate emergency response department.
- **FR-06** The system shall automatically assign the most suitable unit within the relevant departments for a case.
- **FR-07** The system shall display all the cases on a single live dashboard.
- **FR-08** The system shall update the status of each case throughout the lifecycle of the case.
- **FR-09** The system shall automatically update a unit's availability status when it is assigned or the case is released.
- **FR-10** The system shall notify the assigned case with its details upon assignment.
- **FR-11** The system shall allow the dispatcher to mark a case (resolved/closed), which releases the unit assigned to it.
- **FR-12** The system shall allow the dispatcher to search through the history of logs by typing date, caller, and ID.

2.2 Non-Functional Requirements

Performance (P)
1. NFR-PO1: When the dispatcher has assigned a case to the system, the system shall send it to the most suitable department for that case in under 2 seconds. 2. NFR-PO2: The system shall allow at least 100 concurrent users without performance issues
Maintainability (M)
1. NFR-MO1: The system shall be built using a modular architecture, so if one module fails, it will not cause the entire system to crash. 2. NFR-MO2: The system shall be designed in a way that allows new user roles and departments to be added without requiring modification of existing classes.
Reliability (R)
1. NFR-RO1: Data from a case shall not be lost when it is being sent from the dispatcher to the selected emergency department. 2. NFR-RO2: The system shall run at least 99.9% of the time, given that it operates in a life-critical environment 24 hours a day. 3. NFR-RO3: All case log entries and incident reports submitted through the system shall be saved without losing any of the data entered. 4. NFR-RO4: When there is a large number of cases open simultaneously, the system shall continue to function without crashing.
Security (S)
1. NFR-SO1: Any data that passes between users and the system must be encrypted to prevent unauthorised access. 2. NFR-SO2: The system shall restrict access to system features based on each user's assigned role, as defined in section 1.1.

Usability (U)
1. NFR-U01: The system shall display a tailored view of the system for each user role, displaying only the tools and information required for that role. 2. NFR-U02: The system shall return all error messages in plain English so they can be easily understood. 3. NFR-U03: All users shall be able to operate the core functions of their role within the system after no more than 1 hour of training.

2.3 Analysing Requirements Verification

The functional and non-functional requirements were analysed using the requirements quality criteria for **clarity, completeness, consistency, correctness, feasibility, and testability**. These criteria are used to identify whether each requirement is clearly defined, consistent with the rest of the system, representative of the intended behaviour, and can be objectively tested.

The following table identifies the main issues found within the current requirements:

Requirement	Quality Criteria	Analysis
FR-01	Clarity, Completeness, Testability	The problem is the lack of clarity for the term “incident notice” The type of the incident could be clearer; for example, is it a type, address, or notification? Thus affecting the completeness, testability, and clarity of this functional requirement.

FR-02	Completeness, Testability	This functional requirement is self-explanatory, and my people would interpret that the dispatcher would be allowed to assign a case from the caller. Although this could be expanded on what the case is assigned to. Therefore, it lacks completeness in describing what it's assigned to, as well as makes it less testable because you don't know the outcome.
FR-03	clarity, completeness, consistency	The word manage is a bit ambiguous in terms of what the system does with the unit's information. Manage could mean just allowing the dispatcher to view, or does it also update the unit's details? This creates an inconsistency issue with the other functional requirement FR-09. Since the boundaries of what counts as "managing", it is not testable since testers wouldn't know whether FR-03 meant to only be viewable or include update behaviours.
FR-04	Completeness, Testability, correctness	This requirement is clear in that most people would interpret it as the system assigning priority levels automatically. However, the term "severity" is not clear, as there are no defined criteria or scale for what determines an incident's severity. This causes a completeness issue, which in turn decreases testability, since without defined severity levels, a tester cannot verify whether a case was assigned the correct priority. This lack of definition also affects correctness, as it cannot be confirmed whether the requirement accurately represents the intended behaviour without knowing how severity is meant to be determined.
FR-05	Completeness, Testability, Correctness	This is clear in terms of that the case would automatically route to a certain emergency response department, but how it is routed to the emergency response department is the question. This decreases the completeness of what the functional requirement would try to comprehend, which decreases its testability of which case gets routed to a certain department. This uncertainty affects the correctness.

FR-06	Completeness, Testability, Correctness	This has the same issue as FR-05, except this is for the individual unit. It is clear that the system automatically assigns a unit within a department, but it does not identify under what circumstances that a unit is the most suitable; it could be availability, proximity, experience, personnel count, or other factors. Therefore diminishes its completeness and testability due to unclear testing criteria, thereby lowering its correctness.
FR-07	completeness, Testability, Correctness	The issue in this requirement is what's displayed on the dashboard and what "live" means. It is clear that the system needs a dashboard to display all cases, but what exactly does each case display on the dashboard? Also, what live means could be a constant update or just updated when the page is refreshed. Both ambiguities reduce the completeness of the requirement, in turn decreasing its testability and correctness.
FR-08	Completeness, testability, correctness	From the requirement, it is clear that the status of the case would change, but what exactly is in the lifecycle of each case? It could be unassigned, in progress, completed, or delayed. This reduces its completeness due to unclear understanding of status changes, which also affects its testability. Both of these issues would result in a decrease in correctness.
FR-09	Completeness, testability, correctness	The case availability status is clear, and there are clear possible outputs of status, although the term "released" is unclear. Does "released" mean that the case is no longer active for the unit, or does the case complete its mission of saving/solving the case? This reduces its completeness and testability due to the unclear status of the unit's availability. This affects the requirements' correctness.
FR-10	Completeness, correctness, testability	Most people would know that this requirement would notify the case, which does not make sense, as the case is not a person; it would need to notify the case of a unit when the unit is assigned a case. Therefore, it is fundamentally not correct and incomplete in what the requirement is trying to convey. This affects its testability, as the requirement is completely false.

FR-11	Consistency	This requirement clarifies what FR-09 does not elaborate on: while FR-09 states that the unit's availability updates when the case is released, it does not specify what action triggers the "release". FR-11 supplies this missing trigger, as the dispatcher manually marks the case as resolved/closed. Since FR-09 does not reference FR-11, there is an inconsistency problem.
FR-12	Completeness, testability, correctness	The requirement clearly explains that the system should allow the dispatcher to search through the history logs but is less clear on how the history logs are searched, even though date, caller, and ID are stated. It could be clearer on how this information contributes to finding the correct call log. For example, whether you need all these info to search for a log or just enter any of these info to get a list of logs. The requirement is incomplete, which in turn makes it difficult to test. This also lessens its correctness; it's hard to confirm whether the search behaviour is intended.
NFR-Po1	Completeness, Correctness, Testability	There is a clear 2-second duration mentioned; however, the requirement is missing the processing workflow. It does not specify whether this duration includes database persistence of cases, network issues that may occur, or decision-making time by the system. The overall correctness and testability are compromised, and testers cannot verify if routing to a department within 2 seconds meets the stability rule.
NFR-Po2	Clarity, Completeness, Testability	"Performance Issues" is ambiguous. Without explicit details like acceptable response times or latency thresholds, it lacks clarity and completeness. This reduces testability, as performance testers have no criteria to determine the pass and fail conditions during load testing with 100 users.

NFR-Mo1	Completeness, Testability	This requirement states an architectural goal, which is “modular architecture”. However, this lacks complete criteria explaining how different issues like fault tolerance should be handled (microservices). This is vague to non-technical stakeholders and diminishes testability because testers need failure scenarios to verify that a failing part of the system doesn’t cause system-wide failure.
NFR-Mo2	Completeness, Testability	The requirement clearly states the Open/Closed Principle, but lacks completeness because it doesn’t define the architectural pattern constraints that are expected. This reduces testability because verifying this requirement requires architectural compliance auditing.
NFR-Ro1	Completeness, Testability	This requirement states that “data should not be lost”, which lacks completeness. It doesn’t specify the fault tolerance required, e.g. persistent message queues, or recovery behaviour during network outages. This impacts testability, as cases like connection issues cannot be deterministically verified.
NFR-Ro2	Clarity, Completeness, Testability	This requirement specifies a 99.9% uptime, which is measurable, but it lacks clarity and completeness regarding boundaries. It doesn’t define whether planned maintenance work counts towards downtime, or over what timeframe the 99.9% uptime is measured (week, month, year), or what critical parts of the system must be active to count the system as “running”. This uncertainty reduces testability.
NFR-Ro3	Completeness, Testability, Correctness	This requirement is focused on no data loss for submissions but lacks completeness on storage conditions. Without specifying validation rules or confirming what counts as “submitted”, testers cannot verify data completeness, impacting the overall correctness of the system’s persistence.
NFR-Ro4	Clarity, Completeness, Testability	In this requirement, the part “large number of cases” is ambiguous. Without a specific threshold, e.g. 1000 cases, the requirement is not complete or clear. This severely impacts testability, as the quality teams cannot establish a baseline for stress and load testing.

NFR-S01	Completeness, Testability	This requirement states that data must be encrypted, but is not complete due to missing the cryptographic standards or key expectations for encryption. Without defining these baselines, testers cannot verify whether the implementation satisfies the required level of security.
NFR-S02	Completeness, Consistency, Testability	This requirement establishes role-based access. However, testability and completeness rely on section 1.1. If this section fails to define the permissions, it can introduce consistency gaps, preventing testers from validating the access boundaries.
NFR-U01	Completeness, Testability	The requirement mentions role-tailored views, enhancing the user's experience. However, it lacks completeness, missing the specific mappings per role. Without a table showcasing exactly what UI components belong to which user role, UX testers cannot test for correctness or verify whether unauthorised UI elements are correctly hidden.
NFR-U02	Clarity, Completeness, Testability	The term "plain English" in this requirement is subjective and can be interpreted differently for each person. This requirement lacks clarity and completeness because it doesn't define the standards or error-handling guidelines. This lack of criteria makes it difficult to test this requirement.
NFR-U03	Testability, Completeness	This requirement states a "1-hour training limit", which provides a clear metric, but it is incomplete on the evaluation parameters. It doesn't define what the core functions of their role are or what the success rate counts as "complete". Without these criteria, usability testing cannot be conducted.

2.4 Improved and more testable requirements

fwe

Task 3: Proposed Solution and Initial Prototype

- ☐ Explain how the proposed solution addresses the selected real-world problem.
- ☐ Develop an initial prototype that demonstrates the main intended functionality.
- ☐ Include a GUI if the solution involves user interaction, data entry, workflow processing, search, reporting, dashboards, or end-user decision support.
- ☐ If a GUI is not appropriate, justify this decision and provide an alternative interface such as an API, command-line tool, automated testing framework, backend service, or QA dashboard.

The proposed project will develop a prototype Emergency Dispatch System. This system will include an interface for the dispatcher to be able to record the logs of each call and assign a case. It will also be able to manage all dispatched units showing their status, location, and other important information. This addresses the usability issue for the dispatcher, and the interface will also address the reliability issue. The system will also include a standardised priority allocator as well as a scheduler, this makes it so priority of each case is automated and standardised so human error is less likely, improving overall reliability of the system. The system will also be more modular in order to improve the overall maintainability when new features have to be added, such as units. This will be addressed through design patterns, and SOLID principles. Software quality should consider whether the product has suitable quality attributes, whether the process uses the effective practices, and whether the system meets all the users needs.

Task 4: AI-Assisted Development Using GitHub Copilot

- ☐ Use GitHub Copilot to support prototype development and QA activities.
- ☐ Identify which parts of the prototype or QA artefacts were supported by AI suggestions.
- ☐ Provide examples of prompts, comments, code suggestions, generated snippets, or AI-assisted outputs.
- ☐ Identify any limitations, errors, risks, bias, security issues, maintainability issues, or quality concerns found in AI-assisted code.
- ☐ Reflect on how AI-assisted coding supported learning, productivity, testing, documentation, or quality awareness.

Task 5: Initial Test Strategy and Test Planning

- ☐ Define the scope of testing and out-of-scope items.
- ☐ Identify relevant test levels or test types, such as unit testing, integration testing, system testing, acceptance testing, usability testing, regression testing, or security testing.
- ☐ Explain the functional testing approach.
- ☐ Explain the non-functional testing approach where relevant.
- ☐ Define entry criteria and exit criteria.
- ☐ Identify the test environment, tools, browsers, devices, platforms, or dependencies.

Task 6: Initial Test Cases and Requirements Traceability

- ☐ Design at least eight (8) initial test cases.
- ☐ Ensure the test cases cover key functional requirements and selected non-functional requirements where appropriate.
- ☐ Create an initial requirements traceability matrix linking requirements to test cases.
- ☐ Explain how traceability supports quality assurance and change management in the project.

Task 7: Project Progress, Risks, and Next Steps

- ☐ Summarise work completed so far.
- ☐ Identify current limitations, risks, and challenges.
- ☐ Explain how risks will be managed in the next phase.
- ☐ Provide a clear plan for completing the final project.
- ☐ Include a brief team reflection on AI-assisted learning, coding, testing, quality assurance, and group collaboration.